# Vulnerability Report

**Archivo:** OrdersController.cs

**Code Analyzed:**

```
?using Azure;
using FermaOrders.API.Controllers.Response;
using FermaOrders.Application.Dto.Application;
using FermaOrders.Application.Dto.Components.Customer;
using FermaOrders.Application.Interface.Application;
using FermaOrders.Domain.Entities;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using System.Net;

namespace FermaOrders.API.Controllers.Application
{
    [Route("api/[controller]")]
    [ApiController]
    public class OrdersController : ControllerBase
    {
        private readonly IOrderService _orderService;

        public OrdersController(IOrderService orderService)
        {
            _orderService = orderService;
        }

        [Authorize(Roles = "Admin,Empleado")]
        [HttpPost]
        [ProducesResponseType(StatusCodes.Status200OK)]
        [ProducesResponseType(StatusCodes.Status403Forbidden)]
        [ProducesResponseType(StatusCodes.Status500InternalServerError)]
        public async Task<IActionResult> CreateOrderAsync([FromBody] OrderDto orderDto)
        {
            var response = new RespuestaAPI();
            try
            {
                if (orderDto == null)
                {
                    return BadRequest("Invalid order data.");
                }

                var order = await _orderService.CreateOrderAsync(orderDto); // Aquí el await
                response.Result = order;
                response.IsSuccess = true;
                response.StatusCode = HttpStatusCode.OK;
                return Ok(response);
            }
            catch (Exception ex)
            {
                response.StatusCode = HttpStatusCode.InternalServerError;
                response.IsSuccess = false;
                response.ErrorMessages.Add($"Error: {ex.Message}");
                return StatusCode(500, response);
            }
        }

        [Authorize(Roles = "Admin,Empleado")]
        [HttpGet("orderitems/{orderId}")]
        [ProducesResponseType(StatusCodes.Status200OK)]
        [ProducesResponseType(StatusCodes.Status403Forbidden)]
        [ProducesResponseType(StatusCodes.Status500InternalServerError)]
        public async Task<IActionResult> GetOrderItemsByOrderIdAsync(int orderId)
        {
            var respuesta = new RespuestaAPI();
            try
            {
                var items = await _orderService.GetOrderItemByOrderAsync(orderId);

                respuesta.StatusCode = HttpStatusCode.OK;
                respuesta.IsSuccess = true;
                respuesta.Result = items;
```

```
                return Ok(respuesta);
            }
            catch (KeyNotFoundException ex)
            {
                respuesta.StatusCode = HttpStatusCode.NotFound;
                respuesta.IsSuccess = false;
                respuesta.ErrorMessages.Add($"Error: {ex.Message}");
                return NotFound(respuesta);
            }
            catch (Exception ex)
            {
                respuesta.StatusCode = HttpStatusCode.InternalServerError;
                respuesta.IsSuccess = false;
                respuesta.ErrorMessages.Add($"Error: {ex.Message}");
                return StatusCode(500, respuesta);
            }
        }


        [Authorize(Roles = "Admin,Empleado")]
        [HttpGet("customers/{customerId}")]
        [ProducesResponseType(StatusCodes.Status200OK)]
        [ProducesResponseType(StatusCodes.Status403Forbidden)]
        [ProducesResponseType(StatusCodes.Status500InternalServerError)]
        public async Task<IActionResult> ListOrderByCustomer(int customerId)
        {
            var respuesta = new RespuestaAPI();
            try
            {
                var orders = await _orderService.ListOrderByCustomer(customerId);

                respuesta.StatusCode = HttpStatusCode.OK;
                respuesta.IsSuccess = true;
                respuesta.Result = orders;

                return Ok(respuesta);
            }
            catch (KeyNotFoundException ex)
            {
                respuesta.StatusCode = HttpStatusCode.NotFound;
                respuesta.IsSuccess = false;
                respuesta.ErrorMessages.Add($"Error: {ex.Message}");
                return NotFound(respuesta);
            }
            catch (Exception ex)
            {
                respuesta.StatusCode = HttpStatusCode.InternalServerError;
                respuesta.IsSuccess = false;
                respuesta.ErrorMessages.Add($"Error: {ex.Message}");
                return StatusCode(500, respuesta);
            }
        }
    }
}
```

**Analysis:** ```html
Security Vulnerabilities

#### Potential Mass Assignment

**Type:** Mass Assignment Vulnerability
**Approximate Line:** Likely within the `_orderService.CreateOrderAsync(orderDto)` method (called on line 40), depending on how the `OrderDto` is mapped to the `Order` entity.
**Description:** The `OrderDto` is passed directly to the service layer, and subsequently used to create/update an `Order` entity. If the `OrderDto` contains properties that should *not* be directly mapped (e.g., `IsAdmin`, `CreatedBy`, or other sensitive fields), an attacker could potentially manipulate these properties by crafting a malicious `OrderDto`.
**Mitigation:** Use a Data Transfer Object (DTO) pattern and explicit mapping. The controller should receive the `OrderDto`, then the controller or application service layer should map *only* the necessary and safe properties from the DTO to the `Order` entity. Do *not* directly pass the DTO to the persistence layer or ORM for creation/update

without sanitizing or mapping fields.
**Code Quality Improvement:** Enforce clear separation of concerns between API models and domain models. Reduce coupling by avoiding direct dependencies between the API layer and the data access layer.

#### Lack of Input Validation/Sanitization

**Type:** Input Validation Vulnerability
**Approximate Line:** Specifically the `CreateOrderAsync`, `GetOrderItemsByOrderIdAsync` and `ListOrderByCustomer` methods (lines 34, 57, and 82).
**Description:** There's a lack of explicit validation and sanitization of the `orderDto`'s properties, `orderId`, and `customerId`. For example, the code doesn't check for negative values, excessively large values, or potentially malicious strings in the order details. This could lead to data integrity issues, denial-of-service attacks, or potentially even SQL injection if the service layer isn't properly parameterized.
**Mitigation:** Implement robust input validation within the controller. Use data annotations in the `OrderDto` (e.g., `[Required]`, `[Range]`, `[StringLength]`) and check `ModelState.IsValid` in the controller actions *before* passing the data to the service layer. For integer parameters (`orderId`, `customerId`), ensure they are positive and within reasonable bounds. Sanitize string inputs to prevent potential injection attacks (though parameterization is generally preferred).
**Code Quality Improvement:** Promotes better data quality, reduces the risk of unexpected errors, and improves the overall reliability of the application.

#### Potential for Information Disclosure through Error Messages

**Type:** Information Disclosure
**Approximate Line:** Error handling blocks in each controller method (lines 44, 67, 92).
**Description:** The code catches exceptions and includes the raw exception message in the `ErrorMessages` list, which is then returned in the API response. In a production environment, exposing detailed exception messages can reveal sensitive information about the application's internal workings (e.g., database schema, file paths, etc.) to potential attackers.
**Mitigation:** Log the full exception details for debugging purposes, but return generic, user-friendly error messages to the client. Create a custom error handling middleware to globally manage exception handling and avoid exposing sensitive information.
**Code Quality Improvement:** Improves the security posture of the application and provides a better user experience by presenting informative but non-revealing error messages.

Code Quality Metrics and Improvements

#### Complexity

**Issue:** Moderate. The controller methods are relatively straightforward, but the try-catch blocks increase cyclomatic complexity.
**Improvement:** Reduce complexity by centralizing exception handling using a global exception filter or middleware. This would remove the duplicated try-catch blocks in each controller method.

#### Duplication

**Issue:** Significant duplication in error handling logic across all controller actions. The try-catch blocks and the construction of the `RespuestaAPI` object are repeated in each method.
**Improvement:** Implement a global exception filter or middleware to handle exceptions in a centralized manner. This will eliminate the duplicated try-catch blocks and the redundant code for constructing error responses. Also consider creating a helper method or base class for building consistent API responses to avoid redundant code.

#### Readability

**Issue:** Good, but could be improved. Consistent naming conventions (e.g., using `response` consistently instead of mixing `response` and `respuesta`) can improve readability. The error handling could be made more concise.
**Improvement:** Adopt consistent naming conventions (English is generally preferred for international teams). Use more descriptive variable names where appropriate. Extract common operations into helper methods to improve code clarity. Use meaningful comments sparingly to explain complex logic. Remove unnecessary comments.

#### Coupling

**Issue:** Moderate coupling between the controller and the `IOrderService`. The API layer directly depends on the service layer. The strength of the DTO to entity relationship can also imply coupling (as noted above).
**Improvement:** Use interfaces to decouple the controller from concrete implementations of the service layer. Ensure a clean separation of concerns. Employ the DTO pattern and explicit mapping (as mentioned above) to reduce tight coupling between the API layer and the domain model. Review your DI configuration, ensure it is only registered once.

Proposed Solution

1. **Implement Input Validation:** Add data annotations to the `OrderDto` and use `ModelState.IsValid` in the controller to validate the input before passing it to the service layer. Add bounds checking to integer parameters.
2. **Implement Sanitization:** Sanitize string inputs if needed, though parameterization is generally preferred to prevent injection attacks.
3. **Address Mass Assignment:** Map properties from the `OrderDto` to the `Order` entity explicitly in the controller or application service layer, rather than passing the DTO directly to the data access layer.
4. **Centralize Exception Handling:** Implement a global exception filter or middleware to handle exceptions in a consistent manner across all controller actions. Return generic error messages to the client while logging detailed exception information for debugging.
5. **Refactor for Reduced Duplication:** Extract common code (e.g., API response construction) into helper methods or a base class to reduce duplication.
6. **Ensure Consistent Naming:** Adopt consistent naming conventions throughout the codebase (e.g., using English consistently).
7. **Dependency Injection review:** Make sure components are only registered once.

```
```